

DOCUMENT TECHNIQUE — DÉVELOPPEUR

Procédure de déploiement

Mettre la plateforme en ligne pour un client : serveur, HTTPS, sauvegardes et mises à jour.

Ce document décrit, étape par étape, comment mettre en ligne la plateforme pour un client : de la commande du serveur jusqu'à la boutique accessible en HTTPS, avec sauvegardes automatiques et procédure de mise à jour.

Durée estimée : 2 à 3 heures pour la première installation, puis environ 1 heure pour chaque nouveau client.

Les fichiers de configuration cités (`deploy/nginx.conf` , `deploy/deploy.sh` ...) sont fournis dans le projet et ont été testés. Remplacez partout `boutique-client.com` par le vrai nom de domaine du client.

Sommaire

1. [Ce qu'il faut obtenir du client](#)
2. [Choisir l'hébergement](#)
3. [Préparer le serveur](#)
4. [Installer les logiciels](#)
5. [Créer la base de données](#)
6. [Installer l'application](#)
7. [Configurer le fichier `.env` de production](#)
8. [Finaliser l'installation de Laravel](#)
9. [Configurer Nginx et le HTTPS](#)
10. [Tâches en arrière-plan et planifiées](#)
11. [Sauvegardes](#)
12. [Vérifications avant de livrer](#)
13. [Mettre à jour la boutique](#)
14. [Plusieurs clients sur un même serveur](#)
15. [Dépannage](#)
16. [Annexe : hébergement mutualisé \(cPanel\)](#)

1. Ce qu'il faut obtenir du client

Faites remplir cette liste **avant** de commencer :

Élément	Exemple	Utilisé pour
Nom de la boutique	« Maison Akwaba »	<code>SHOP_NAME</code>
Nom de domaine	<code>boutique-client.com</code>	Adresse du site
Logo (SVG ou PNG fond transparent) + couleurs	—	Identité visuelle

Élément	Exemple	Utilisé pour
E-mail, téléphone, WhatsApp, adresse, horaires	—	Page contact, pied de page
Réseaux sociaux	Facebook, Instagram...	Pied de page
Numéros Mobile Money marchands + nom du titulaire	Orange, MTN, Moov, Wave	Paielement par transfert
Livreur partenaire : site, téléphone, WhatsApp	—	Bouton « Contacter un livreur »
Modes et tarifs de livraison	Abidjan 2 000 F, intérieur 5 000 F	Admin › Livraisons
Adresse e-mail d'envoi + accès SMTP	commandes@boutique-client.com	E-mails aux clients
Textes légaux validés (CGV, confidentialité, retours)	—	Pages d'information
Produits : noms, prix, descriptions, vraies photos	Fichier Excel + dossier photos	Catalogue
E-mail du propriétaire (compte administrateur)	—	Premier compte

2. Choisir l'hébergement

Option recommandée : un serveur VPS

Un VPS est un petit serveur loué, sur lequel vous avez tous les droits. C'est la solution la plus fiable pour cette plateforme : base PostgreSQL, tâches en arrière-plan, HTTPS gratuit.

Critère	Minimum conseillé
Système	Ubuntu 24.04 LTS
Mémoire	2 Go de RAM (4 Go à partir de 3 boutiques sur le même serveur)
Disque	40 Go SSD
Prix indicatif	5 à 12 € par mois (Hetzner, Contabo, OVHcloud, DigitalOcean, Hostinger VPS...)

Choisissez un centre de données proche de vos utilisateurs : en Europe (France, Allemagne) pour l'Afrique de l'Ouest si aucun fournisseur local fiable n'est disponible.

Nom de domaine

Achetez-le chez un registraire (OVH, Namecheap, Gandi...) ou un registraire local pour un `.ci`. Il coûte environ 10 à 20 € par an pour un `.com`.

Dans la zone DNS du domaine, créez **deux enregistrements A** pointant vers l'adresse IP du VPS :

Type	Nom	Valeur
A	@	IP.DU.SERVEUR
A	www	IP.DU.SERVEUR

La propagation DNS prend de quelques minutes à quelques heures.

Alternative : hébergement mutualisé (cPanel)

C'est moins cher, mais plus limité. Voir [l'annexe](#).

3. Préparer le serveur

Depuis PowerShell sur votre PC, connectez-vous au serveur avec l'IP et le mot de passe `root` envoyés par l'hébergeur :

```
ssh root@IP.DU.SERVEUR
```

Mettez le système à jour et créez un utilisateur dédié, `deploy`. On ne fait jamais tourner un site en `root`.

```
apt update && apt upgrade -y
adduser deploy                # choisir un mot de passe fort
usermod -aG sudo deploy
```

Activez le pare-feu : seuls SSH, HTTP et HTTPS restent ouverts.

```
apt install -y ufw
ufw allow OpenSSH
ufw allow 80
ufw allow 443
ufw enable
```

Déconnectez-vous (`exit`), puis reconnectez-vous avec le nouvel utilisateur :

```
ssh deploy@IP.DU.SERVEUR
```

Recommandé : configurez ensuite une connexion par clé SSH et désactivez la connexion `root` par mot de passe.

4. Installer les logiciels

```
# PHP 8.4 et ses extensions
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:ondrej/php
sudo apt update
sudo apt install -y php8.4-fpm php8.4-cli php8.4-pgsql php8.4-gd php8.4-intl \
    php8.4-mbstring php8.4-xml php8.4-curl php8.4-zip php8.4-bcmath

# Serveur web, base de données, outils
sudo apt install -y nginx postgresql git unzip supervisor certbot python3-certbot-nginx

# Composer (gestionnaire de paquets PHP)
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

# Node.js 22 (pour compiler le CSS et le JavaScript)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
```

Vérifiez :

```
php -v && composer -V && node -v && psql --version && nginx -v
```

Faites tourner PHP sous l'utilisateur `deploy`. Cela évite tous les problèmes de droits sur les fichiers :

```
sudo sed -i 's/^user = www-data/user = deploy/; s/^group = www-data/group = deploy/'
/etc/php/8.4/fpm/pool.d/www.conf
sudo systemctl restart php8.4-fpm
```

5. Créer la base de données

Générez un mot de passe fort et **notez-le** :

```
openssl rand -base64 24
```

```
sudo -u postgres psql
```

```
CREATE USER boutique WITH PASSWORD 'LE_MOT_DE_PASSE_GENERE';
CREATE DATABASE boutique OWNER boutique;
\q
```

6. Installer l'application

```
sudo mkdir -p /var/www/boutique
sudo chown deploy:deploy /var/www/boutique
cd /var/www/boutique
git clone https://github.com/VOTRE-COMPTE/VOTRE-DEPOT.git current
cd current
git checkout main
```

Dépôt privé : GitHub demande un identifiant. Créez un *Personal Access Token* (GitHub > Settings > Developer settings > Fine-grained tokens, lecture seule sur le dépôt) et utilisez-le comme mot de passe. Vous pouvez aussi configurer une *deploy key* SSH.

Branche : le code doit être fusionné dans `main`. Pour déployer une autre branche : `git checkout nom-de-la-branche`, et `BRANCH=nom-de-la-branche ./deploy/deploy.sh` lors des mises à jour.

Installez les dépendances et compilez les assets :

```
composer install --no-dev --optimize-autoloader
npm ci
npm run build
```

7. Configurer le fichier .env de production

```
cp .env.example .env
php artisan key:generate
nano .env
```

Dans `nano`, modifiez les valeurs suivantes, puis enregistrez avec `Ctrl+O`, `Entrée`, et quittez avec `Ctrl+X`.

```

APP_NAME="Maison Akwaba"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://boutique-client.com

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=boutique
DB_USERNAME=boutique
DB_PASSWORD=LE_MOT_DE_PASSE_GENERE

SESSION_ENCRYPT=true
SESSION_SECURE_COOKIE=true

# E-mails (identifiants fournis par le prestataire d'envoi : Brevo, Mailgun, l'hébergeur
de messagerie...)
MAIL_MAILER=smtp
MAIL_HOST=smtp.prestataire.com
MAIL_PORT=587
MAIL_USERNAME=identifiant
MAIL_PASSWORD=mot-de-passe
MAIL_FROM_ADDRESS="commandes@boutique-client.com"
MAIL_FROM_NAME="${APP_NAME}"
SHOP_ADMIN_NOTIFICATION_EMAIL=proprietaire@boutique-client.com

# Identité et contact
SHOP_NAME="${APP_NAME}"
SHOP_TAGLINE="Le slogan du client"
SHOP_CONTACT_EMAIL=contact@boutique-client.com
SHOP_CONTACT_PHONE="+225 07 00 00 00 00"
SHOP_CONTACT_WHATSAPP="+225 07 00 00 00 00"
SHOP_SOCIAL_FACEBOOK=https://facebook.com/...
SHOP_SOCIAL_INSTAGRAM=https://instagram.com/...

# Paiements : jamais "sandbox" en production
PAYMENT_GATEWAYS=cash_on_delivery,manual_mobile_money
MOMO_ACCOUNT_NAME="Maison Akwaba SARL"
MOMO_ORANGE_NUMBER="07 00 00 00 00"
MOMO_WAVE_NUMBER="01 00 00 00 00"

# Livreur (modifiable ensuite par le client dans Admin > Paramètres)
COURIER_NAME="Nom du livreur partenaire"
COURIER_URL=https://plateforme-du-livreur.com
COURIER_WHATSAPP="+225 07 00 00 00 00"

```

Protégez le fichier : il contient des mots de passe.

```
chmod 600 .env
```

Logo et favicon du client : copiez ses fichiers dans `public/images/brand/`, puis indiquez-les dans `.env` : `SHOP_LOGO=images/brand/logo.svg` et `SHOP_FAVICON=images/brand/favicon.png`.

Couleurs : elles se modifient dans `resources/css/theme.css` . Faites-le de préférence sur votre PC, dans une branche dédiée au client, puis relancez `npm run build` sur le serveur.

8. Finaliser l'installation de Laravel

```
php artisan migrate --force           # crée les tables
php artisan db:seed --force           # rôles + modes de livraison par défaut (aucune
donnée de démo en production)
php artisan storage:link              # rend les images accessibles
php artisan shop:create-admin         # compte du propriétaire (mot de passe de 12
caractères minimum)
php artisan optimize                  # met en cache la configuration, les routes et les
vues
```

Ne lancez **jamais** `migrate:fresh` ni `DemoSeeder` sur un serveur de production : ces commandes effacent ou remplissent la base avec des données fictives. Le `DemoSeeder` refuse d'ailleurs de s'exécuter en production.

9. Configurer Nginx et le HTTPS

```
sudo cp deploy/nginx.conf /etc/nginx/sites-available/boutique
sudo sed -i 's/DOMAINE/boutique-client.com/g' /etc/nginx/sites-available/boutique
sudo ln -s /etc/nginx/sites-available/boutique /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t                      # doit afficher « syntax is ok »
sudo systemctl reload nginx
```

À ce stade, `http://boutique-client.com` affiche la boutique, mais sans HTTPS. Activez le certificat gratuit Let's Encrypt (le DNS doit déjà pointer vers le serveur) :

```
sudo certbot --nginx -d boutique-client.com -d www.boutique-client.com
```

Répondez aux questions (e-mail, acceptation des conditions). Certbot configure HTTPS, la redirection automatique et le renouvellement du certificat.

Ce que la configuration Nginx fournie fait pour vous : elle sert uniquement le dossier `public/` ; elle bloque l'accès à `.env` , `.git` , `vendor/` et aux fichiers PHP autres que `index.php` ; elle met les images et le CSS en cache 30 jours ; elle autorise des uploads jusqu'à 12 Mo.

10. Tâches en arrière-plan et planifiées

Worker de file d'attente (envoi des e-mails)

```
sudo cp deploy/supervisor-worker.conf /etc/supervisor/conf.d/boutique-worker.conf
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl status          # doit afficher RUNNING
```

Planificateur (expiration des paiements, nettoyage)

```
mkdir -p /var/www/boutique/backups
crontab -e          # choisir nano si demandé
```

Collez le contenu de `deploy/crontab.txt` à la fin du fichier, puis enregistrez.

11. Sauvegardes

Le script `deploy/backup.sh`, lancé chaque nuit à 3 h par la tâche cron ci-dessus, sauvegarde la base de données et les images dans `/var/www/boutique/backups`, et conserve 14 jours d'historique.

Testez-le une fois à la main :

```
./deploy/backup.sh
ls -lh /var/www/boutique/backups
```

Important : une sauvegarde stockée sur le même serveur ne protège pas contre la perte du serveur. Copiez régulièrement le dossier `backups` ailleurs : sur votre PC (`scp -r deploy@IP:/var/www/boutique/backups .`) ou sur un stockage externe. Activez aussi, si possible, les sauvegardes automatiques proposées par l'hébergeur du VPS.

Restaurer une sauvegarde :

```
pg_restore -h 127.0.0.1 -U boutique -d boutique --clean --no-owner
/var/www/boutique/backups/db_AAAA-MM-JJ_HHMM.dump
tar -xzf /var/www/boutique/backups/media_AAAA-MM-JJ_HHMM.tar.gz -C
/var/www/boutique/current/storage/app
```

12. Vérifications avant de livrer

Cochez chaque point :

- ☐ `https://boutique-client.com` s'affiche avec le cadenas HTTPS ; `http://` et `www.` redirigent correctement.
- ☐ Connexion à `/admin` avec le compte du propriétaire.
- ☐ Admin > Paramètres : coordonnées, textes d'accueil, livreur renseignés.
- ☐ Admin > Livraisons : modes et tarifs du client.
- ☐ Catégories et premiers produits créés, avec de vraies photos.
- ☐ **Commande test complète** avec un compte client : panier, commande, réception de l'e-mail de confirmation.
- ☐ Le propriétaire a reçu l'e-mail « nouvelle commande ».
- ☐ Dans l'admin : confirmer la commande, tester « Contacter un livreur », passer la commande jusqu'à « Livrée », puis annuler ou supprimer la commande test.
- ☐ Page d'erreur : `https://boutique-client.com/page-inexistante` affiche une belle page 404, sans message technique.
- ☐ `https://boutique-client.com/robots.txt` et `/sitemap.xml` répondent. Soumettez le sitemap dans Google Search Console.
- ☐ Affichage vérifié sur téléphone.
- ☐ Sauvegarde testée (section 11).
- ☐ Remise au client du **guide d'utilisation** (`docs/GUIDE-UTILISATEUR.pdf`) et de ses identifiants.

13. Mettre à jour la boutique

Après avoir modifié et testé le code sur votre PC, puis poussé sur GitHub :

```
ssh deploy@IP.DU.SERVEUR
cd /var/www/boutique/current
./deploy/deploy.sh
```

Le script met le site en maintenance quelques secondes, récupère le code, installe les dépendances, compile les assets, applique les nouvelles migrations, reconstruit les caches, redémarre le worker et remet le site en ligne.

Faites toujours une sauvegarde (`./deploy/backup.sh`) avant une mise à jour importante.

14. Plusieurs clients sur un même serveur

Pour chaque client, répétez les étapes avec des noms distincts :

Élément	Client 1	Client 2
Dossier	<code>/var/www/boutique/current</code>	<code>/var/www/client2/current</code>

Élément	Client 1	Client 2
Base de données	boutique	client2 (utilisateur et mot de passe propres)
Fichier Nginx	sites-available/boutique	sites-available/client2 (autre domaine, autre chemin root)
Worker	boutique-worker.conf	client2-worker.conf (adapter [program:] et les chemins)
Cron	lignes pointant vers /var/www/boutique/...	mêmes lignes vers /var/www/client2/...
Scripts	./deploy/deploy.sh	APP_DIR=/var/www/client2/current ./deploy/deploy.sh

Un VPS de 4 Go de RAM supporte confortablement plusieurs petites boutiques.

15. Dépannage

Symptôme	Cause probable	Solution
Page « 500 / Une erreur est survenue »	Erreur applicative	Lire storage/logs/laravel.log (tail -50 storage/logs/laravel.log)
« Permission denied » dans les logs	Droits sur storage/	sudo chown -R deploy:deploy /var/www/boutique puis vérifier l'étape 4 (PHP sous deploy)
Modification du .env sans effet	Configuration en cache	php artisan optimize
Images produits absentes	Lien storage manquant ou mauvais APP_URL	php artisan storage:link, vérifier APP_URL
Page sans style (CSS absent)	Assets non compilés	npm ci && npm run build
Les e-mails ne partent pas	Worker arrêté ou SMTP incorrect	sudo supervisorctl status, tail storage/logs/worker.log, vérifier MAIL_*
Erreur 419 « Session expirée »	Cookie sécurisé alors que le site est en HTTP	Activer HTTPS (étape 9) ou SESSION_SECURE_COOKIE=false temporairement
Upload d'image refusé (413)	Fichier trop lourd	Augmenter client_max_body_size dans Nginx et upload_max_filesize / post_max_size dans /etc/php/8.4/fpm/php.ini
Derrière Cloudflare : boucle de redirection ou liens en http://	Proxy non déclaré	TRUSTED_PROXIES=* dans .env, puis php artisan optimize ; Cloudflare en mode SSL « Full (strict) »

Annexe : hébergement mutualisé (cPanel)

C'est possible, mais avec des limites : pas de worker permanent, et PostgreSQL rarement disponible. La plateforme fonctionne aussi avec **MySQL / MariaDB**, testée.

1. **Prérequis chez l'hébergeur** : PHP 8.3 ou plus récent, accès « Terminal » ou SSH dans cPanel, possibilité de définir le dossier racine du domaine.
2. **Sur votre PC** : `composer install --no-dev --optimize-autoloader` et `npm run build`, puis compressez le projet en ZIP, **sans** `node_modules`.
3. **cPanel › Gestionnaire de fichiers** : envoyez le ZIP dans un dossier hors de `public_html` (ex. `~/boutique`) et décompressez-le.
4. **cPanel › Domaines** : faites pointer la **racine du domaine** vers `~/boutique/public`. C'est indispensable : sinon `.env` serait accessible depuis internet.
5. **cPanel › Bases de données MySQL** : créez une base et un utilisateur avec tous les privilèges.
6. `.env` : comme à l'étape 7, avec `DB_CONNECTION=mysql`, `DB_PORT=3306`, la base créée, et `QUEUE_CONNECTION=sync` (les e-mails partent immédiatement, sans worker).
7. **Terminal cPanel** : `cd ~/boutique` puis les commandes de l'étape 8.
8. **cPanel › Tâches Cron** : chaque minute, `cd ~/boutique && php artisan schedule:run >> /dev/null 2>&1`.
9. **SSL** : activez le certificat gratuit (AutoSSL / Let's Encrypt) dans cPanel.
10. **Sauvegardes** : utilisez l'outil de sauvegarde de cPanel.